Quality function deployment ensures that the new design in an enterprise IT architecture project fully implements functional and nonfunctional requirements so that planned IT systems can support the business.

Murat Erder and Pierre Pureur

QFD in the Architecture Development Process

Consider the following scenario: A loan-servicing organization is developing an architecture to enable expansion into new markets outside of its traditional domain. The first project launched is a new online loan application system. The organization implements this system, but it is riddled with performance problems due to the organization's lack of experience with the new technology it chose for the application. To make matters worse, while the IT group is still implementing this new application and integrating it with the core mainframe system, management initiates a massive new billing and payments project. Management contends that this project is the core of the business and the organization must implement a new system to enable business growth. However, the architects have decided to build this application on a totally different technology set than the loan application system. After spending millions of dollars and hiring several consulting companies, the organization now supports three major technology sets (loan application, billing/payments, and core applications on mainframe). None of the applications performs as expected. Also, the organization has not achieved expected growth in loans because of resources it diverted to the billing/payments system, which the organization could have used for other enabling systems that would increase market share.

What went wrong? How can the organization link and prioritize its IT initiatives to the actual business? How can the organization make sure that the architectural solutions chosen are complementary to one another?

Here, we outline an approach to designing architectures that fully supports requirements, by leveraging a technique called quality function deployment. QFD is a systematic method for tying product and service design decisions directly to customer wants and needs.

Inside

The Architect's View of QFD

WHY ARCHITECTURE?

When we talk about enterprise IT architecture, we are specifically referring to the IT concerns of a large enterprise. The most common reasons to develop an enterprise IT architecture are as follows:

- *Define the organization's guiding principles and standards.* The architecture is a vision of the future, with the supporting tools to help you get there.
- *Develop blueprints.* In this case, the architecture is concerned with abstracting at an appropriate level to make business and technical decisions.
- *Build common services.* The architecture focuses on defining interfaces.
- *Create a road map to an IT future state.* The architecture deals with transition planning activities that lead to successfully implementing an IT blueprint.

Effectively and efficiently transforming an entity's operational and technological environ-

 Published by the IEEE Computer Society 1520-9202/03/$17.00 © 2003 IEEE

ment requires a blueprint, which we commonly refer to as an enterprise architecture. Such an architecture includes descriptive models (defined in business and technology terms) to aid decision-makers in understanding the complexities regarding how the entity operates today and how it wants to operate in the future. It also includes prescriptive models designed to help organizations successfully transition to their desired IT environment.

One of the key enterprise architecture challenges is to provide traceability between business drivers (such as faster time to market, improving quality or service, or controlling costs) and architectural decisions.

LEVERAGE QFD TO LINK SYSTEM BENEFITS AND REQUIREMENTS

Like many manufacturing organizations that cater to consumer markets, IT organizations must respond to their users' changing demands while simultaneously supporting ongoing operations. As with many larger manufacturing organizations, the complexity of the delivery system (which in the case of IT means applications, data, and IT infrastructure) does not lend itself to rapid change without significant capital investment.

To effectively manage these challenges, IT organizations need to adopt the same type of engineering-oriented mindset that has proven so effective in other types of businesses where success is tightly coupled to effective design and the use of technology.

QFD helps in translating customer needs into technical requirements. Initially deployed within the automobile industry in the 1970s, QFD has spread to multiple global industries, including the software engineering industry. Hewlett-Packard and AT&T have successfully applied QFD to software development.

The QFD technique is based on a simple concept: It enables transformations from a prioritized set of drivers to a set of candidate enablers, ensuring that the requirements can be traced throughout the architecture design.

As a simple example of QFD usage, assume you are designing a new system for a nonprofit entity that organizes international sporting events. Your team has interviewed the key users who have expressed desired benefits for the new system. Those benefits, in order of priority, are

- better service for the athletes—for example, better response to participants' queries, requests for special accommodations, and requests for special transportation needs between the event venues and the airport;
- faster response to problems that could arise during an event;
- improved service for the officials; and
- increased integrity for the sporting events through increased authentication of athletes' credentials.

Assume also that your team has collected a list of functional requirements for the system (use cases), which include

- obtaining a day pass on the spot for athletes, so they could attend several events on a given day when they are not participating in any events themselves;
- checking accreditation of athletes, officials, and other participants;
- accommodating issues such as special requests for computer or telecommunications equipment as the athletes check in at their accomodations; and
- auctioning tickets to the public, in case some tickets remain unsold right before an event.

The problem is that your users can't agree on the priorities of their requirements, and the budget allocated to the project isn't sufficient to implement all requirements. You need to select two of those four requirements for implementation. How do you achieve this goal?

Our solution is to leverage QFD, linking the benefits to the requirements. Figure 1 shows the results of this process.

In Figure 1 and all of this article's QFD examples, the numbers have the following meaning:

- The numbers in the weight column represent the weights assigned to each benefit.
- The numbers in the rank column represent the relative rank of each benefit, based on its weight.
- The numbers in the correlation to benefits row represent the result of the QFD calculation—in this case, benefit weight times relationship weight based on the relationships (strong, moderate, or weak) assigned during the QFD process. The usual relationship weights are 9 for a strong relationship, 3 for a moderate relationship, and 1 for a weak relationship.
- The red numbers in the rank row represent the relative rank of each requirement (output list from the QFD process) based on the correlation to benefits numbers.

Figure 1. Simple QFD example.

Key:
- ● Strong relationship
- ▽ Moderate relationship
- ○ Weak relationship

		1 Accommodate issues	2 Auction tickets	3 Obtain day pass	4 Check accreditation	Weight	Rank	
Better service to games participants	1	●	○		○	10.0	1.0	1
Faster response to problems	2	●				9.0	2.0	2
Better service to officials	3	●	○	○		9.0	2.0	3
Better integrity for the games	4	○			○	7.0	4.0	4
Correlation to benefits	1	273	57	27	51			
Rank	2	1	2	4	3			
		1	2	3	4			

The Architect's View of QFD

The architect sees QFD as a quality assurance technique for translating customer needs into technical requirements. Typically, an architect will use QFD to

- **list customer requirements and their importance;**
- **identify implementation possibilities for those requirements, with alternatives;**
- **input requirements and implementation possibilities into a QFD house of quality, implemented through an Excel spreadsheet or specialized QFD software;**
- **determine interactions between implementation possibilities (the roof of the house of quality in Figure 2);**
- **determine how much each implementation possibility contributes to the realization of each requirement, proceeding column by column (most QFD practitioners use the following weighting scheme: strong = 9, moderate = 3, weak =1);**
- **perform calculations and rank implementation possibilities; and**
- **reexamine implementation possibilities based on the result of this process, and arrive at the possibility (or combination of alternatives) that best fulfills the requirements.**

QFD is unique in its emphasis on the voice of the customer and efforts to combine qualitative and quantitative data. QFD facilitates development of a common vision among all stakeholders regarding the product under construction by keeping them actively engaged throughout the project. Iterative use of QFD throughout the life cycle maximizes the degree to which the value of the end product will be evident to the target audience.

One of the greatest advantages of QFD is that it clarifies the customer requirements and the rationale for architecture decisions. However, please keep the following in mind:

- **Many assignments with weights for contributions are subjective. You can best perform this process in a workshop where every participant has an equal vote. However, the process can be quite lengthy.**
- **A good QFD tool is a prerequisite for handling real-life architecture situations, where you might have to deal with various requirements and implementation possibilities. An Excel spreadsheet might be sufficient for a simple QFD exercise. However, this could soon prove frustrating as the number of requirements increases. We have used QFD/capture from International TechneGroup Inc (ITI) over the last five years and are reasonably happy with it, but other tools exist on the market. Figure A shows a QFD/Capture tool sample output.**

You can combine QFD with other software system design

- The numbers in the far right column also represent the rank of each benefit or requirement.

The QFD technique shows "accommodate issues" as the most important requirement to satisfy, followed by "auction tickets," "obtain day pass," and "check accreditation." Our decision is therefore to implement "Accommodate issues" and "auction tickets" in the first release of the system.

From this simple example, you can see that the QFD technique lets architects correlate how well the design criteria will meet the customer needs. "The Architect's View of QFD" sidebar shows how architects typically use QFD. We rank the relationships here as *strong*, *moderate*, or *weak*. (We didn't use weak relationships in our earlier simplified example of the nonprofit entity that organizes international sporting events.) We group these relationship types in a relationship matrix, also known as *house of quality*, as Figure 2 shows.

By leveraging this tool (house of quality), the design team can identify neglected needs and find incorporated design features that might not necessarily meet any needs.

An additional QFD feature addresses the fact that not all features can coexist when seeking to meet customer requirements. It helps to have a tool that not only correlates design elements to the customer requirements but also performs a trade-off correlation between realization mechanisms to ensure that conflicting design elements such as centralized and decentralized databases are not present. A positive correlation indicates that two design elements complement each other. A negative correlation denotes a possible interference between the implementation of these design elements. Realization mechanism correlations are stored in the "roof" of the house of quality, as Figure 2 shows.

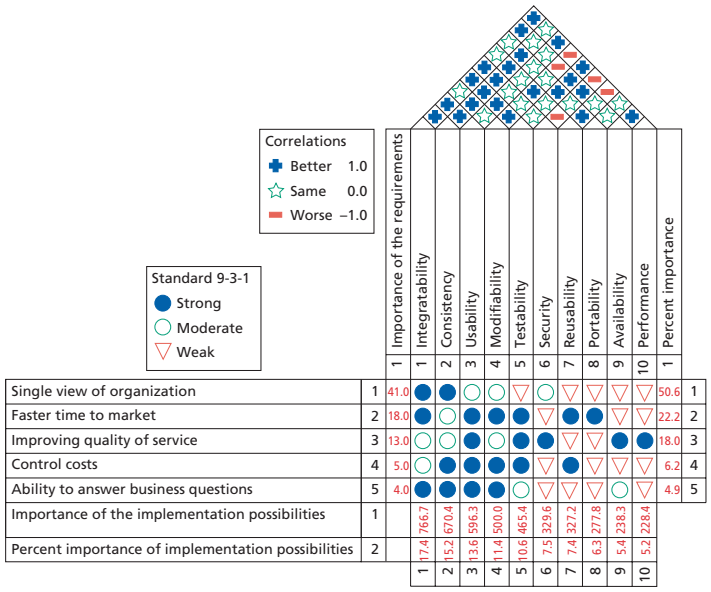
The house of quality tool includes other optional components, such as the following:

- *A competitive assessment of customer needs*. The QFD team uses this to assess the team's customer needs against its perceived competition.
- *A competitive assessment of technical features*. This ranks the product's technical features against the competition.
- *Probability factors*. We use these to indicate the ease of implementation of the product's technical features.

techniques such as Design Space to create rigorous approaches to system design (M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Discipline*, Prentice Hall, 1996). American Management Systems (AMS) cofounder John Hauser, professor of marketing at the Massachusetts Institute of Technology Sloan School of Management, coauthored the first widely read English paper about QFD (J. Hauser and D. Clausing, "The House of Quality," Harvard Business School Press, 1988).

Several excellent books discuss leveraging QFD for product development (L.R. Guinta and N.C. Praizler, *The QFD Book: The Team Approach to Solving Problems and Satisfying Customers through Quality Function Deployment*," AMACOM, 1993) and using QFD for software development (G. Herzwurm, S. Schockert and W. Mellis, "*Joint Requirements Engineering: QFD for Rapid User-Focused Software Development*," Friedrich Vieweg & Sohn, 2000).

Figure A. Sample output from the QFD/Capture tool from ITI.



In summary, think of QFD as a structured mechanism for documenting justifications for decisions and assumptions that architecture design process participants must make.

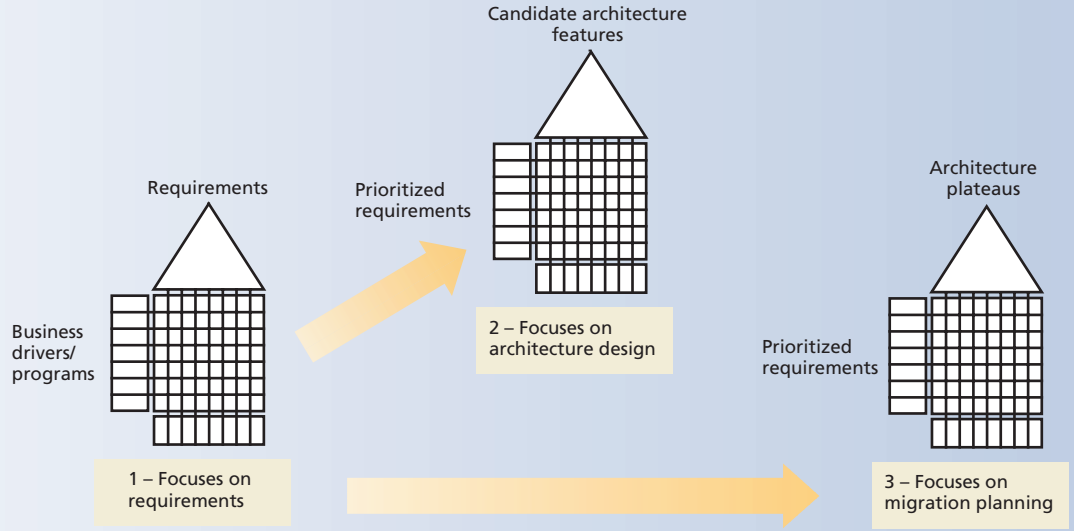
QFD IN ACTION: THREE SCENARIOS

The architect can use QFD often during the architecture development process. Some of the most common usage scenarios include the following:

- *Prioritizing user requirements.* This use of QFD occurs early in the architecture process, usually at the conclusion of the requirements capture phase, in which you identify the various stakeholders' functional and nonfunctional architecture requirements. Using QFD, the architecture team correlates the architecture requirements to the key business drivers. The team can also use QFD in this scenario to prioritize requirements against the programs that business units plan or execute to implement business strategies. The team can also use QFD in this scenario to clarify the business program priorities by correlating the programs to the business vision and strategies. Finally, the team might leverage QFD to prioritize the architecture principles by also correlating

Figure 2. QFD house of quality.

Figure 3. Leveraging QFD during architecture design.



them to the business vision and strategies.

- *Translating these requirements into an architecture design.* This type of QFD usage concerns prioritizing and selecting design choices. Prioritized requirements from the requirements prioritization scenario serve as inputs to this process. The team prioritizes and selects the architecture designs and features by relating them to the QFD input set. Similarly, the team selects technology products—databases, operating systems, and application servers—by correlating them to the QFD input set. Correlating them to the architecture principles can further refine both selections of designs, features, and technology products.
- *Planning architecture releases (transition planning).* The third QFD usage category involves sequencing architecture rollout phases. Once again, the prioritized architecture requirements from the requirements prioritization scenario serve as inputs. The team prioritizes and selects each architecture platform's components to be rolled out in sequence, to ensure a smooth transition between the current and the future IT states by correlating them against the QFD input set.

Figure 3 depicts these scenarios.

Using QFD to Prioritize Architecture Requirements

Our first scenario uses QFD to link the architecture requirements to the business drivers. One of the input parameters to the QFD process is a set of priorities for the business drivers, and the QFD process lets the team prioritize the requirements against these priorities.

A simple example illustrates this scenario. Assume you are in the requirements phase of an enterprise architecture project for a financial services institution (we will call it firm X). Your architecture team has gathered a list of business drivers. As the following list indicates, your team has prioritized the business drivers and has given each driver a weight; with 50 being the maximum weight. The higher the weight is, the more important the driver:

- Control costs (weight 5), such as the cost of delivering and operating new application systems;
- Increase time to market (weight 18);
- Enable customers (and yourself) to have a single view of firm X (weight 41);
- Improve quality of service (weight 13);
- Improve the financial services institution's ability to answer questions from business users (weight 4).

Your team has also collected a list of potential nonfunctional architecture requirements, as Table 1 shows.

The team can now organize a requirements prioritization workshop with the key stakeholders. During this workshop, the team asks the stakeholders to review the list of business drivers, confirm their assigned priorities, and review the list of candidate nonfunctional requirements for completeness.

It is now time to use the QFD tool. Before the workshop, the team inputs the business drivers, their weights, and the candidate nonfunctional requirements into the tool, and a team member's laptop computer screen projects the QFD house of quality. This team member will act as the scribe for the exercise and will capture the exercise results in real time, providing instant feedback to the workshop attendees.

For each requirement in every column, the group asks,

Table 1. Potential nonfunctional architecture requirements.

Nonfunctional requirement	Level	Architecture issues
Performance	Runtime	Intercomponent communication; Dividing functionality to exploit parallelism
Security	Runtime	Specialized components (secure kernels, firewalls, and authentication servers)
Availability	Runtime	Fault tolerance, redundant components, and controlling component interactions
Usability	Runtime	Linked to modifiability. Important cost-control quality. Error handling, error avoidance, and user satisfaction
Consistency	Runtime	Ability for the system to return consistent results when used by several clients
Modifiability	Build	Modularized, encapsulated components
Portability	Build	Portability layer
Reusability	Build	Loose coupling between components
Integratability	Build	Compatible interconnection mechanisms, consistent interfaces, "uses" relationship (where one system component leverages the functionality that another component provides), allows incremental builds
Testability	Build	Modularized, encapsulating components

"Does this requirement contribute to the realization of this driver strongly, moderately, weakly, or not at all?" As the group agrees on a contribution level, the scribe records it by using the appropriate symbol, as Figure 4 shows. It is critical that the group address each column, at least for the first few requirements, until the workshop attendees are comfortable with the process.

When all relationships are determined, the system automatically calculates the rankings for the requirements (as Figure 4 shows). This is an iterative process, and the workshop leader should explain this to the participants before the workshop attendees start determining the relationships. This iterative approach lets the participants reach agreement faster on the contribution levels. No decision is final and you can revisit decisions later. The iterative approach also lets the team review and adjust the relationships after the first pass is completed. In practice, more than two iterations are seldom required, and teams make few adjustments during the second iteration.

When reviewing the results of the QFD exercise, the clustering of requirements is more important than their ranking, so the team should review the score of each requirement in relation to the others. For the example in Figure 4, security, reusability, and portability would fall into one cluster, whereas availability and performance would fall into a less-important cluster.

In a short period of time, your team has managed to elicit

Figure 4: Using QFD for prioritizing nonfunctional architecture requirements.

We used standard weights to prioritize each quality against the drivers. The output of the process is a prioritized list of nonfunctional requirements.

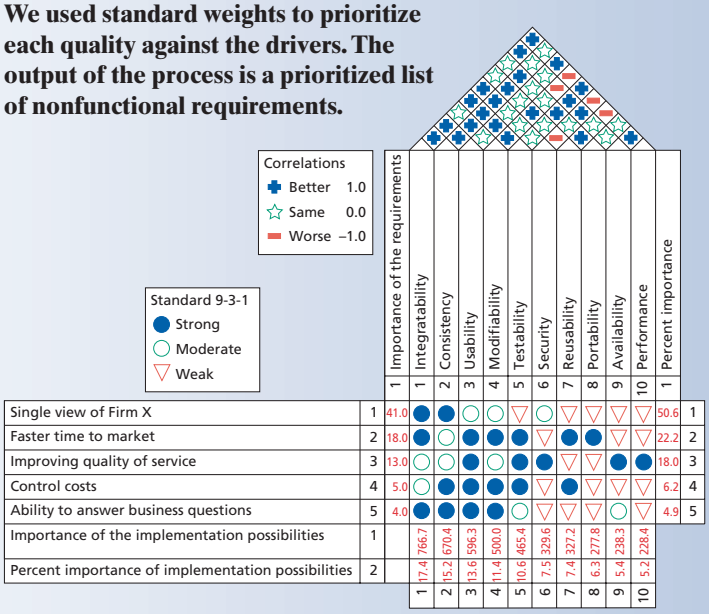


Figure 5. Using QFD for prioritizing architecture design features.

Standard 9-3-1
● Strong 1.0
○ Moderate 0.0
▽ Weak –1.0

		1 Common messaging standards	2 Intelligent (integration) bus	3 XML/SOAP protocol	4 CAB data store with common API	5 Securities master with common API	6 SOA–industry D/B I/F service	7 SOA–ADP interfaces service	8 SOA–security depository I/F service	9 SOA–customer interface service	10 Consolidated logical trade data store	11 SOA–Trade validation service	12 SOA–Trade compression service	13 SOA–exception processing	1 Importance of the functional requirements	2 Percent of importance	
Trade data pre-screened or validated for exception processing prior to execution	1	●	○	●	●	●	○			○	●	●		○	●	4.2	1
Customer order/trade data electronically captured and fed into an STP environment	2	●	●	○	○	●	▽			○	●	●	○	○	●	4.2	2
Pre-screened trade data routed through industry databases such as Bloomberg for enrichment	3	●	○	●	▽	○	●				●	●			●	4.2	3
Trade status information maintained and distributed through trade management system	4	●	●	○	▽	○	▽		○		●	●	●		●	4.2	4
Common repository of trade information	5	●	○	●	●	●	○				●	○	●	○	●	4.2	5
Real-time STP environment	6	●	●	○	●	●	●	●	●	●	●	●	●	●	●	4.2	6
Orders placed electronically into a centralized order management system on a real-time basis	7	●	●	●	●	○				○	○	○			●	4.2	7
Orders managed through a centralized order management system	8	●	●	○	○	○			○		○	○		○	●	4.2	8
Standardized order processes across products	9	●	○	●	○	○	▽		○		○	○	●	○	●	4.2	9
Electronic trade tickets	10	○	▽	○	●	○				○	▽	▽		○	●	4.2	10
Trade processing standardized across products wherever possible. Detailed procedures created by product when necessary	11	●	▽	●	○	●		○			●	○	○		●	4.2	11
All steps of execution automated	12	○	●	○	○	○	○	●	●	●	○	○	▽	●	●	4.2	12
Trades input into trade processing system without time lag from original execution	13	○	●	○	○	○	○	○		▽	▽	●			●	4.2	13
Electronic tickets routed to operations for input	14	○	○	○	○	○					▽	○	▽		○	1.4	14
Execution data including price, quantity, and counter-party captured electronically in real time	15	●	○	○	●	●	○			▽	○	●			●	4.2	15
All trades matched to internal sources in an automated manner for tracking and audit trail	16	●	●	●	○	●			○		●	▽			○	1.4	16
Electronic matching of salesperson and trader sides of trade	17	●	▽	●	○	○					○	▽			○	1.4	17
Automated feeds to settlement systems	18	○	●	▽	○	○	▽	●	●		●	▽			●	4.2	18
Automated crossing engines	19	●	●	●	●	●	○		●		●				●	4.2	19
Automated order routing based on liquidity and best execution	20	●	●	○	○	●	▽				○				○	1.4	20
Automated links to clearing agents and clearing houses	21	○	▽	○	▽	○	▽		●		○				●	4.2	21
Automated linkage of street side transaction to customer side confirmation	22	○	▽	○	●	○				●	○				●	4.2	22
Allocation and split instructions captured automatically and verified for processing	23	▽	●	○	●	○	▽			○	▽				○	1.4	23
Trade management system handles all trade states	24	●	▽	○	▽	●					●				●	4.2	24
Exception-based processing	25	●	●	●	●	●					○	●			●	4.2	25
Centralized confirmation factories used for settlement activities	26	●	●	●	●	●					▽				●	4.2	26
Electronic confirmations	27	○	▽	○	●	○				●	○				●	4.2	27
Electronic receipt of allocation instructions	28	○	▽	○	○	○				○	○				○	1.4	28

business drivers and nonfunctional requirements, and prioritize them. More importantly, the architecture stakeholders feel that they had a key input into the process, and decisions are fully traceable—all because of their use of QFD.

Using QFD to Design the Architecture

Our second scenario concerns prioritizing and selecting design choices. Using QFD, the team selects the design components that are best suited to fulfill the architecture requirements.

Let's return to our previous example to illustrate this scenario. Your architecture team at firm X has progressed to the project's design phase, and the team has gathered and prioritized a list of functional requirements, as Figure 5 shows. As the figure also shows, our team defined some candidate design features for the new architecture.

The team must now link the candidate design features to the requirements. The team uses QFD in a workshop similar to the requirements prioritization workshop with the key stakeholders that we discussed earlier in the Using QFD to Prioritize Architecture Requirements section. Figure 5 shows the results of the QFD workshop.

As a result of this workshop, the team decides to focus on designing the new architecture around common messaging standards, an integration bus, and XML-based protocols.

Using QFD to Create the Architecture Transition Plan

Our third scenario for illustrating the use of QFD during the architecture process involves sequencing architecture rollout phases. Using QFD, the team selects the components of each architecture plateau to be rolled out in sequence.

Again, let's look at a simple example to illustrate this scenario. Assume you are in the architecture transition planning phase of a wireless architecture project for our

earlier example of the nonprofit entity that organizes and manages international sporting events. This phase's objective is to schedule the implementation of several wireless solutions (which we selected and defined during the architecture design phase) to optimize their effectiveness. Your architecture team has gathered and prioritized a list of functional requirements (documented as use cases). The system must

- handle issues,
- communicate via wireless devices,
- communicate with national coordinators,
- transport participants,
- notify you of change of venue,
- access information from wireless devices,
- check in at the athletic facility,
- access the Internet at the athletic facility,
- perform medical checks for antidoping,
- perform bookkeeping (cash, payroll, and rate cards),
- access the information system from kiosks,
- auction tickets,
- obtain a day pass,
- check accreditation, and
- check in staff members at venues.

Wireless solutions that will undergo implementation in our example as a result of this project include

- leveraging cellular phones to provide instant messaging capabilities;
- wireless enabling of a help desk system, information system, and auction Web site; and
- using wireless LANs, mobile quality assurance, and advanced access control.

Your team now needs to group those solutions into plateaus. They decide to use QFD in a workshop similar to the requirements prioritization workshop with the key stakeholders that we discussed earlier. Figure 6 shows the QFD workshop's results.

Again, the clustering of wireless solutions is more important than their absolute ranking. From the results of this exercise, the team defines the following implementation plateaus:

- *Plateau 1.* Leverage cellular phones (instant messaging).
- *Plateau 2.* Enable wireless help desk system and wireless information system.
- *Plateau 3.* Establish wireless LANs and mobile quality assurance.
- *Plateau 4.* Provide advanced access control and enable wireless auction Web site.

Figure 6. Using QFD to create an architecture transition plan.

Key:
- ● Strong relationship
- ▽ Moderate relationship
- ○ Weak relationship

		1 Leverage cellular phones	2 Wireless enablement of help desk system	3 Wireless enablement of information system	4 Wireless LANs	5 Mobile quality assurance	6 Access control	7 Wireless enablement of auction Web site	1 Correlation to benefits	2 Rank
Correlation to use cases	1	2128.2	850.8	606.9	482.4	401.4	183.6	121.5		
Handle issues	1	●	●						426	1
Communicate via wireless devices	2	●	○	○					382	2
Communicate with national coordinators	3	●	○	○					375	3
Transport participants	4	●	▽						340	4
Notify change of venue	5	●	○	○					339	5
Access information system from wireless devices	6	●		●					309	6
Check in at the athlete facility	7	○	○			●			300	7
Access Internet at the athlete facility	8				●				249	8
Perform medical checks for antidoping	9	○	▽			●			146	9
Perform bookkeeping (cash, payroll, rate cards)	10				●				146	10
Access information system from kiosks	11				●				141	11
Auction tickets	12	○						●	135	12
Obtain day pass	13						●		93	13
Check accreditation	14						●		72	14
Check in staff at venues	15						●		39	15
Rank	1	1.0	2.0	3.0	4.0	5.0	6.0	7.0		

The stakeholders review and accept this definition of architecture plateaus, and the team proceeds with the detailed scheduling of each plateau's implementation.

Based on our experience in delivering successful enterprise architecture projects for companies in several industries and countries, we have discovered that this approach yields the following advantages over more traditional process-driven approaches:

- You can identify and address key customer needs.
- You can prioritize business requirements early in the process, and drive all design and implementation planning activities.
- Business stakeholders fully participate in the architecture design process.
- All architecture design decisions are linked to business requirements.
- The team tends to get the enterprise architecture right the first time, instead of endlessly trying to retrofit missed requirements into a finished product.
- The overall cost of the project is lower, because fewer iterations are necessary to create an architecture design that satisfies user requirements.

Going back to the case study involving the loan servic-

ing organization from the beginning of this article, we believe that this project's outcome could have been very different if the project team had adopted an approach similar to the one we are outlining here. Instead of having to support several disparate technology platforms and deal with the associated costs and staffing problems, the team could have created a homogeneous solution set that fully supported their business and leveraged a common technology infrastructure. Ensuring that key business requirements drive architecture design is a critical success factor for enterprise architecture initiatives.

In the future, we plan to explore issues associated with defining and collecting architecture requirements, designing efficient and elegant architectures, and creating architecture transition plans. ■

Acknowledgments

We thank the staff at BearingPoint, especially Manuel Barbero, managing director and head of technology consulting for BearingPoint's Financial Services, for the valuable support they provided for the writing and associated research of this article.

Murat Erder *is a senior manager at BearingPoint in London. Contact him at murat.erder@bearingpoint.com.*

Pierre Pureur *is a director at BearingPoint in Stamford, CT. Contact him at ppureur@bearingpoint.net.*

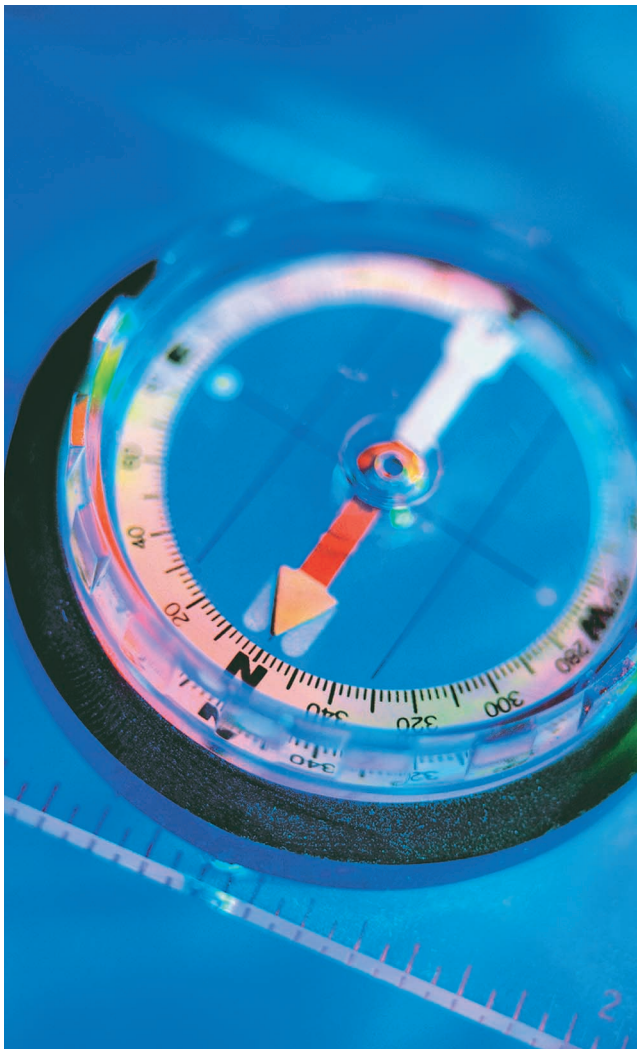
For more information on this or any other computing topic, visit our Digital Library at http://computer.org/publications/dlib.

Look to the Future

IEEE Internet Computing reports emerging tools, technologies, and applications implemented through the Internet to support a worldwide computing environment.

In the next year, we'll look at
- Identity Management
- Business Processes for the Web
- Internationalizing the Web
- Internet-Based Data Dissemination
- the Wireless Grid
- Measuring Performance
- Homeland Security

... and more!

IEEE Internet Computing

www.computer.org/internet